

- Relationship of the component with other components and packages
- Time stamp of SBOM creation

SBOM standards

To make organisations adopt SBOM quickly, several standards have been defined for generating them. A defined schema provides a format for listing software components, and the most widely used are Software Package Data Exchange (SPDX) and CycloneDX.

SPDX: SPDX is an SBOM format developed by the Linux Foundation. It shows relationships between software components, licences, and copyrights associated with the software. It can be stored in different file formats, such as SPDX and JSON. It defines three elements: documents (metadata about the SBOM), packages (groups of elements), and files (single files).

CycloneDX: This lightweight SBOM standard is maintained by the OWASP community and built on top of SPDX. It gives security-related information such as supplier, manufacturer, target components, etc. It can be stored in different file formats, such as XML and JSON. It gets divided into components, services, and dependencies.

Docker, too, has recently introduced an SBOM format for container images. It has announced the *docker sbom CLI* command in Docker Desktop 4.7.0 as a CLI plugin, which lists all the components of the container image. It is an experimental functionality for now, which has been developed as an open source collaboration with Anchore through the Syft tool.

Syft is a tool for generating SBOMs from container images and file systems. It can convert SBOMs to different formats.

Installation of the Docker CLI plugin

- As a pre-requisite, Docker/Docker Desktop must be installed on your system. Upgrade the Docker desktop if its version is less than 4.7.0.
- Install the *docker sbom* plugin.

```
curl -sSfL https://raw.githubusercontent.com/docker/sbom-cli-plugin/main/install.sh | sh -s -
```

- Check its version and verify installation:

```
docker sbom version
```

Note: If you are using Docker as a CLI tool, then before installing *docker sbom* create a */root/.docker* directory. More information can be found at <https://github.com/docker/sbom-cli-plugin/issues/25>.

Different Docker SBOM commands

- To check all options related to docker sbom, type:

```
docker sbom --help
```

- To list the software components of an NGINX Docker image, type:

```
docker sbom nginx:latest
```

By default, the output will be in a table format. From the output given below, it can be inferred that the first image is pulled if not present locally, then it's loaded and processed further, and then the software elements get listed along with

```
root@master:~# docker sbom --help
Usage: docker sbom [OPTIONS] COMMAND

View the packaged-based Software Bill of Materials (SBOM) for an image.

EXPERIMENTAL: The flags and outputs of this command may change. Leave feedback on https://github.com/docker/sbom-cli-plugin.

Examples:
  docker sbom alpine:latest              a summary of discovered packages
  docker sbom alpine:latest --format syft-json  show all possible cataloging details
  docker sbom alpine:latest --output sbom.txt  write report output to a file
  docker sbom alpine:latest --exclude /lib --exclude '**/*.db'  ignore one or more paths/globs in the image

Options:
  -D, --debug                show debug logging
  --exclude stringArray      exclude paths from being scanned using a glob expression
  --format string            report output format, options=[syft-json cyclonedx-xml cyclonedx-json github-json spdx-tag-value spdx-json table text] (default "table")
  --layers string            [experimental] selection of layers to catalog, options=[squashed all] (default "squashed")
  -o, --output string        file to write the default report output to (default is STDOUT)
  --platform string          an optional platform specifier for container image sources (e.g. 'linux/arm64', 'linux/arm64/v8', 'arm64', 'linux')
  -q, --quiet                suppress all non-report output
  -v, --version              version for sbom

Commands:
  version  Show Docker sbom version information

Run 'docker sbom COMMAND --help' for more information on a command.
```

Figure 1: Different sub-commands with *docker sbom*